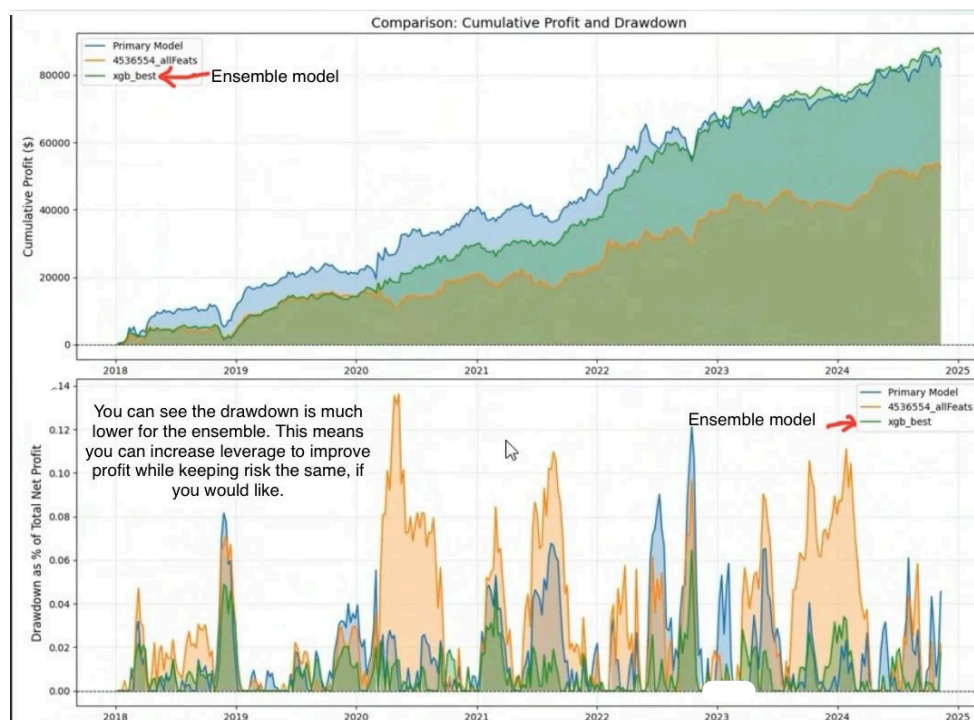


r/algotrading • 1y ago
Yocurt

...

Meta Labeling for Algorithmic Trading: How to Amplify a Real Edge

Education



I've commented briefly on some other posts mentioning this approach, and there usually seems to be some interest so I figured it would be good to make a full post.

There is a lot of misunderstanding and misconceptions about how to use machine learning for algo trading, and unrealistic expectations for what it's capable of.

I see many people asking about using machine learning to predict price, find a strategy, etc. However, this is almost always bound to fail - machine learning is NOT good at creating its own edge out of nowhere (especially LLM's, I see that a lot too. They'll just tell you what it thinks you want to hear. They're an amazing tool, but not for that purpose.)

ML will not find patterns by itself from candlesticks or indicators or whatever else you just throw at it (too much noise, it can't generalize well).

A much better approach for using machine learning is to have an underlying strategy that has an existing edge, and train a model on the results of that strategy so it learns to filter out low quality trades. The labels you train on could be either the win / loss outcomes of each trade (binary classification, usually the easiest), the p/l distribution, or any metric you want, but this means it's a supervised learning problem instead of unsupervised, which is MUCH easier, especially when the use case is trading. The goal is for the model to AMPLIFY your strategies existing edge.

Finding an edge -> ml bad

Improving an existing edge -> ml good

Introduction

Meta labeling was made popular by Marco Lopez de Prado (head of Abu Dhabi Investment fund). I highly recommend his book "Advances in Financial Machine Learning" where he introduces the method. It is used by many funds / individuals and has been proven to be effective, unlike many other ml applications in trading.

With meta labeling, instead of trying to forecast raw market movements, you run a primary strategy first — one that you've backtested and know already has at least a small edge and a positive expectancy. The core idea is that you separate the signal generation and the signal filtering. The primary signal is from your base strategy — for example, a simple trend-following or mean-reversion rule that generates all potential trade entry and exit times. The meta label is a machine learning model that predicts whether each individual signal should be taken or skipped based on features available at the time.

Example: your primary strategy takes every breakout, but many breakouts fail. The meta model learns to spot conditions where breakouts tend to fail — like low volatility or no volume expansion — and tells you to skip those. This keeps you aligned with your strategy's logic while cutting out the worst trades. In my experience, my win rate improves anywhere from 1-3% (modest but absolutely worth it - don't get your hopes up for a perfect strategy). This has the biggest impact on drawdowns, allowing me to withstand downturns better. This small % improvement can be the difference between losing money with the strategy or never needing to work again.

Basic Workflow

1. Run Your Primary Strategy

Generate trade signals as usual. Log each signal with entry time, exit time, and resulting label you will assign to the trade (i.e. win or loss). IMPORTANT - for this dataset, you want to record EVERY signal, even if you're already in a trade at the time. This is crucial because the ML filter may skip many trades, so you don't know whether you would have really been in a trade at that time or not. I would recommend having AT LEAST 1000 trades for this. The models need enough data to learn from. The more data the better, but 5000+ is where I start to feel more comfortable.

[Skip to main content](#)[+ Create](#)[🔔 1](#)

Assign a binary label to each signal: 1 if the trade was profitable above a certain threshold, 0 if not. This becomes your target for the meta model to learn / predict. (It is possible to label based on pnl distribution or other metrics, but I'd highly recommend starting with binary classification. Definitely easiest to implement to get started and works great.) A trick I like to use is to label a trade as a loser also if it took too long to play out ($> n$ bars for example). This emphasizes the signals that followed through quickly to the model.

3. Gather Features for Each Signal

For every signal, collect features that were available at the time of entry. (Must be EXACTLY at entry time to ensure no data leakage!) These might include indicators, price action stats, volatility measures, or order book features.

4. Train the Meta Model

Use these features and labels to train a classifier that predicts whether a new signal will be a win or loss (1 or 0). (More about this below)

5. Deploy

In live trading, the primary strategy generates signals as usual, but each signal is passed through the trained meta model filter, along with the features the model uses. Only signals predicted with over a certain confidence level are executed.

Feature Engineering Tips:

- Use diverse feature types: combine price-based, volume-based, volatility-based, order book, and time-based features to capture
- Prioritize features that stay relevant over time; markets change, so test for non-stationarity and avoid features that decay fa
- Track regime shifts: include features that hint at different market states (trend vs. chop, high vs. low volatility).
- Use proper feature selection: methods like RFECV, mutual information, or embedded model importance help drop useless or redun
- Always verify that features are available at signal time – no future data leaks.

Modeling Approaches:

It's important to balance the classes in the models. I would look up how to do this if your labels are not close to 50-50, there is plenty of information out there on this as it's not unique to meta labeling.

Don't rely on just one ML model. Train several different types — like XGBoost, Random Forest, SVM, or plain Logistic Regression — because each picks up different patterns in your features. Use different feature sets and tune hyperparameters for each base model to avoid all of them making the same mistakes.

Once you have these base models, you can use their individual predictions (should be probabilities from 0-1) to train an ensemble method to make the final prediction. A simple Logistic Regression works well here: it takes each base model's probability as input and learns how to weight them together.

Calibrate each base model's output first (with Platt scaling or isotonic regression) so their probabilities actually reflect real-world hit rates. The final ensemble probability gives you a more reliable confidence score for each signal — which you can use to filter trades or size positions more effectively.

I'd recommend making a calibration plot (image 2) to see if your ensemble is accurate (always on out-of-fold test sets of course). If it is, you can choose the confidence threshold required to take a trade when you go live. If it's not, it can still work, but you may not be able to pick a specific threshold (would just pick > 0.5 instead).

Backtesting Considerations + Common Mistakes

When testing, always compare the meta-labeled strategy to the raw strategy. Look for improvements in average trade return, higher Sharpe, reduced drawdown, and more stable equity curves. Check if you're filtering out too many good trades — too aggressive filtering can destroy your edge. Plotting the equity and drawdown curves on the same plot can help visualize the improvement (image 1). This is done by making one out of sample (discussed later) prediction for every trade, and using those predictions on each trade to reconstruct your backtest results (this removes trades that the model said to skip from your backtest results).

An important metric that I would try to optimize for is the precision model. This is the percentage of trades the model predicted as winners that were actually winners.

Now to the common mistakes that can completely ruin this whole process, and make your results unreliable and unusable. You need to be 100% sure that you prevent/check for these issues in your code before you can be confident in and trust the results.

Overfitting: This happens when your model learns patterns that aren't real — just noise in your data. It shows perfect results on your training set and maybe even on a single test split, but fails live because it can't generalize.

To prevent this, use a robust cross validation technique. If your trades are IID (look this up to see if it applies to you), use nested cross-validation. It works like this:

- You split your data into several folds.
- The outer loop holds out one fold as a true test set – this part never sees any model training or tuning.
- The inner loop splits the remaining folds again to tune hyperparameters and train the model.

[Skip to main content](#)[+ Create](#)[🔔 1](#)

This way, your final test results come from data the model has never seen in any form — no leakage. This is repeated n times for n folds, and if your results are consistent across all test folds, you can be much more confident it is not overfit (never can be positive though until forward testing).

If your trades are not IID, use combinatorial purged cross-validation instead. It's stricter: it removes overlapping data points between training and testing folds that could leak future info backward. This keeps the model from "peeking" at data it wouldn't have in real time.

The result: you get a realistic sense of how your meta model will perform live when you combine the results from each outer fold — not just how well it fits past noise.

Data Leakage: This happens when your model accidentally uses information it wouldn't have in real time. Leakage destroys your backtest because the model looks smarter than it is.

Classic examples: using future price data to build features, using labels that peek ahead, or failing to time-align indicators properly.

To prevent it:

- Double-check that every feature comes only from information available at the exact moment your signal fires. (Labels are the on
- Lag your features if needed – for example, don't use the current candle's close if you couldn't have known it yet.
- Use strict walk-forward or combinatorial purged cross-validation to catch hidden leaks where training and test sets overlap in

A leaked model might show perfect backtest results but will break down instantly in live trading because it's solving an impossible problem with information you won't have.

These two will be specific to your unique set ups, just make sure to be careful and keep them in mind.

Those are the two most important, but here's some others:

- Unstable Features: Features that change historically break your model. Test features for consistent distributions over time.
- Redundant Features: Too many similar features confuse the model and add noise. Use feature selection to drop what doesn't help.
- Too Small Sample Size: Too few trades means model can't learn, and you won't have enough data for accurate cross validation.
- Ignoring Costs: Always include slippage, fees, and real fills. (Should go without saying)

Closing Thoughts:

- Meta labeling doesn't create an edge from nothing — it sharpens an edge you already have. If your base strategy is random, filtering it won't save you. But if you have a real signal, a well-built meta model can boost your risk-adjusted returns, smooth your equity curve, and cut drawdowns. Keep it simple, test honestly, and treat it like a risk filter, not a crystal ball.

Images explained: I am away from my computer right now so sorry the images are the clearest, they're what I had available. Let me try to explain them.

1. This shows the equity curve and drawdown as a % of final value for each backtest. The original strategy with no meta labeling applied is blue, and the ensemble model is green. You can see the ensemble ended with a similar profit as the original model, but its drawdowns were far lower. You could leverage higher each trade while staying within the same risk to increase profits, or just keep the lower risk.
2. This plot shows the change in average trade values (expected per trade) on the y-axis, and the win rate on the x-axis. Each point is a result from an outer test fold, each using different seeds to randomize shuffling, training splits, etc. This lets you estimate the confidence interval that the true improvement from the meta labeling model lies in. In this case, you can see it is 95% confident the average trade improvement is within the green shaded area (average of \$12.03 higher per trade), and the win rate (since I used wins/losses as my labels!) increase is within the yellow shaded area (average of 2.94% more accurate).
3. Example of how a calibration plot may look for the ensemble model. Top horizontal dashed line is the original win rate of the primary models strategy. Lower dashed line is the win rate from the filtered labels based on win/loss and time threshold I used (must have won quicker than n bars...). You can see the win rate for the ensemble model in the green and blue lines, choosing a threshold over either dashed line signifies a win % improvement at that confidence level!

If anyone else has applied this before, I'd love to hear about your experience, and please add anything I might have missed. And any questions or if I could clarify anything more please ask, I'll try to answer them all. Thanks for reading this far, and sorry for the mouthful!

👍 626 🔄 88 🗨️ 2 ➦

Join the conversation

Sort by: Best ▾

🔍 Search Comments



mvstartdevnull • 1y ago

[Skip to main content](#)[+](#) Create[🔔](#) 1

questions:

- On average, how many features are you training your models with?
- Do you use just OHLCV data or do you complement with ticker/orderbook or other datasets?
- Do you have any ALGO's in production, what do returns (& drawdowns ;-)) look like (feel free not to answer) ;-)
- If you have any in production, how? Personally I am considering running them on my homelab Kubernetes cluster for now - ideally colocated at some point. I ask because perhaps there is a way smarter approach
- How do you design your pipeline, any tips? I find myself starting completely from scratch for every strat I come up with which isn't very effective
- Where do you source reliable historic data (for cheap ;-))? Particularly tick and orderbook data seems pretty expensive if you need a bunch of it
- I notice you do not mention deep learning at all, what is your experience with that? LSTM/GRU etc? Or are metalabeled sets usually too small to consider deep learning?
- In your experience, how much performance loss might you expect if you compare bench results (backtesting on unseen test data or walkforward) compared to production?
- Can you give any guidance on realistic slippage settings on ES and NQ?

Any feedback is greatly appreciated man!

30 Reply ...

Yocurt OP • 1y ago

Thanks! Here's what I'd say for those:

- I use six base models, and the features used range from 20-70 for each. Some with the lower range, some with higher range.
- I use mbo data (level 3) from Databento, so I have some orderbook features, as well as some macro based features from FRED.
- just two in production right now, one for NQ one for ES. Both around a 1.15 profit factor and 2.5 calmar ratio.
- I have them coded in Ninjatrade. Definitely better solutions out there but it works for what I need.
- modulate the parts are typically repeated across strategies, then just reuse those parts for the new ones.
- I use Databento, it's probably the best source but a little pricey. Kibot has pretty good quality and much cheaper - good alternative.
- personally I haven't tried deep learning other than some basic ANN's. I feel like it's overkill for many applications, and trading is one of them. YMMV though.
- ideally, if you set everything up right, you will see 0 performance loss compared to your bench results. But you need to give it enough time to play out as well (be sure you did everything 100% correct though, otherwise you'll be waiting forever).
- For ES, 1 tick for entries and exits is usually more than enough. For NQ, 4 ticks is more reasonable. However I use the mbo data, so I can simulate slippage in real time based on the order book, which I would highly recommend.

mvstartdevnull • 1y ago

Amazing, thanks a lot!

If you don't mind, a couple more :-)

- You mentioned applying ML only to capitalize on an existing Edge ... but .. what are your minimum requirements before even considering applying ML?
- How often do you retrain your models in production? Is this time based or do you monitor some metric?
- I am considering using non-binary labels, so as opposed to winner/loser: predict a profit/loss quantile or some risk adjusted metric ... do you have any experience with this?
- The six models you describe, do they all have the same purpose or do you create specialized models for different purposes?
- How are you simulating slippage? Are you replaying the mbo data tick by tick?
- How are you getting your predictions (Python I assume) into Ninjatrade?

Yocurt OP • 1y ago

No problem! Also this first one is a really good question. I typically look for a strategy that was profitable in over ~80% of the years I backtested on, and always do at least five years. Mainly looking for a profit factor of at least 1.07, over ~2500 total trades (enough to be statistically significant), and a max drawdown that's no higher than the average yearly profit. I'm confident in the results from my backtest engine since it models slippage and fees accurately, so these would each be a bit higher otherwise.

For the rest:

- I retrain weekly. Technically every day there's a new signal would be ideal, but if your dataset already has thousands of trades, weekly or even monthly is fine.
- if you use non-binary, what you said would be a multi-class classification problem. Certainly should work fine! Just may be a bit harder to implement. The other approach would be predicting a continuous variable, but I personally hate doing that and find it way harder. Binary or multi-class are both great though!
- I guess slightly different purposes since each has their strengths and weaknesses, and then the ensemble attempts to detect these and take advantage of where each is strongest.
- yeah exactly, pretty much iterating through every single event. Then at trade times, you know the bid/ask prices and their sizes so you know where your fill would be / if you would get filled at all if using a limit order. If you want to do this there's plenty of good tutorials online. I think it's worth the effort so you can actually trust the results.
- I run a python script with a socket that opens a port on my pc, then for every trade signal on Ninjatrade I send the features needed to the script, it makes its prediction, then sends whatever action it decided back to Ninjatrade for it to handle.

mvstartdevnull • 1y ago

**VincentJalapeno** • 1y ago

Are you also running your trades / strategy through Ninjatrade ? Been in the process of basically doing what you have described with running all the data and features in Ninjatrade. Or are you running a separate dashboard outside of Ninjatrade to tell you when to take trades and you execute in Ninjatrade ?

**Yocurt OP** • 1y ago

I did integrate the meta labeling output into my Ninjatrade strategy - I was actually pretty surprised it's able to do this. You can use a socket to open a port on your pc, and run a python script separately to listen on that port. Then at each of the base strategies signals on Ninjatrade, it sends an array of all the feature values at that time to your Python script, then uses the trained model to make a prediction on that array, and sends the output back to Ninjatrade. Then do whatever on Ninjatrade based on that result.

This is nice because it lets you do any preprocessing or that kind of stuff all on Python which in my opinion is much more flexible / easy to use than ninjascript, and then handle all the strategy based logic and entries / exits on Ninjatrade. The latency can be a bit annoying depending on what your doing in the Python script, but it should still be well under 1 second still total which has been fine for me.

**VincentJalapeno** • 1y ago

With the amount of models and data processing you're doing, are you running any kind of processing to be done by the GPU? I do like using the sockets and mostly rely on spreadsheets being created or Ninjatrade itself to give me some details during the process of trades. I have noticed though that certain models have made my Ninjatrade run slower than others, but I assume that's probably somewhere in the optimization of the model itself.

**Yocurt OP** • 1y ago

I do use my gpu for training the base models and the ensemble, but then when running live, these models are all already fully trained so you can just save their weights and load the saved models into your Python script. Then this way, for each trade every model just needs to do inference on that single row which should be extremely fast. Sorry i may not have understood you fully though - does this answer your question?

**VincentJalapeno** • 1y ago

It does ! I appreciate how in-depth and knowledgeable you have been with the post and comments ! This has more or less been my side project the past few months until my motivation died a little, but you have given me inspiration to hit the lab and code again !

**Yocurt OP** • 1y ago

That's great! Ive also had periods where the motivation isn't there, but it's always fun to come back to. This definitely can take some time to implement correctly, but once you have the general outline for it, it is easy to replicate for different strategies. I would make sure any work you do is easy to follow and modular so you can use it easily again in the future and not need to repeat anything every time. There is always time in the future to improve on whatever you get done each time the motivation comes back around, as long as you keep heading in the right direction that's all that matters! Good luck!

**Alexk2468** • 1y ago

can you share exactly how you have this set up? i also have a ninjatrade strategy and i would like to have a similar setup to you

**wallbouncing** • 1y ago

This is probably the most useful post I've seen here in a while with actual real-life information.

**Phunk_Nugget** • 1y ago • Edited 1y ago

Great post.

I've had some success with meta labelling for filtering using LightGBM. Not super far along with it, especially with finding the best features to use, but it made me smile the few times I saw my model's PnL go up while the trade count went down De Prado also mentions using meta labelling for the bet size as well, although I haven't tried that yet, but plan to.

While I generally agree with your statement about finding an edge and ML, I have had some success in combining techniques (and a lot of local compute) to find profitable trade signals without any edge to begin with. I spent years figuring out a way to frame the problem using variations of ideas from AFML and other sources and built everything from scratch in C#/F#, but I do have statistically significant models based on it. That being said, I'm adjusting my direction at the moment, since, as I learn more, I see serious limitations in that approach. It did lead me from trying to scalp to more intraday swing/momentum as a trading style.



Skip to main content

+ Create

1



are so many more variables. Some seem to feature engineering, optimal hyperparameter training, and then you make sure not to have any look-ahead bias, data leakage, etc. I enjoyed your post, thanks.

**Trollsense** • 1y ago

Would suggest you try OSS Vizier for hyperparameter training, found it to outperform Optuna.

<https://research.google/blog/open-source-vizier-towards-reliable-and-flexible-hyperparameter-and-blackbox-optimization/>

**Yocurt OP** • 1y ago

Thank you, and those are good choices! I really like HGB and CatBoost too, they complement the other models well.

**chazzmoney** • 1y ago

When you say HGB do you mean the hgboost project or do you mean Histogram Gradient Boosting?

**xTruegloryx** • 1y ago

HistGradientBoostingClassifier sklearn

**chazzmoney** • 1y ago

Thanks!

**Heyitsmejohn123** • 1y ago

nice detailed post I like it!

**tomdieck** • 1y ago

This is the highest quality post on quant trading on reddit I've ever read.

**Directive31** • 1y ago

this says a lot about this sub... actually about everything I need to know. so thank you for wiping any last doubt off my mind of ever finding anything of value in here.

i'm sorry for what folks on this sub self-inflict and will endure and I wish you well.

Wish mods would find a way to up the quality but it's clearly not happening on reddit...

seeing myself out of this sub now...

**zhatz111** • 1y ago

Great post thanks for this detailed write up! A question for you, how many assets are you trading at once and at what frequency? You mention it is good to have 5000+ trades to feel comfortable so just wondering how you generally hit that.

Also do you train a new ensemble for each asset or generalize it across assets with a specific strategy?

**Yocurt OP** • 1y ago

Thank you! And for my use case, I use it for my day trading strategies on ES and NQ, a separate model for each though. They generate about 3-6 signals a day, so over 5+ years of backtests I get to around that 5000 number.

You could use the same primary base strategy on multiple instruments/assets to increase the number if your strategy doesn't take as many trades, but for this you would always want to include a categorical feature that represents the symbol the signal is for.

**echoeysaber** • 1y ago

For additional context, the meta labelling OP explains is also coined the Triple Barrier Method by Lopez, and he suggests value of 1,0,-1, where 1 is the trade achieves the take-profit threshold (higher price than entry price of the trade for longs), within a set number of observations (price, dollar or volume bar), indicating

[Skip to main content](#) Create 1**Yocurt OP** • 1y ago

Yep exactly! Thank you for clarifying, that is an important point.

**[deleted]** • 1y ago

Nice all good and well if initial strategy isn't over fit in first place

**Yocurt OP** • 1y ago

Haha very true, that's its own beast entirely.

**mvstartdevnull** • 1y ago

Hah, this is exactly what I am experimenting with currently without knowing the actual term for it!

Thanks for the full writeup man, very helpful!

**syd-slice** • 1y ago

This is a very well put post. Thanks

**MackDriver0** • 1y ago

Great article! Thank you very much for sharing.

**blearx** • 1y ago

I currently (after a couple of years, also read AFML and his book after that) have success with the triple barrier method. No meta modeling as of now but the performance is great. I use a pretty extensive process including pretraining as I dont use any tree-based models. The targets I have right now are the yang zhang volatility which can be a bit tight sometimes in margins. I want to try to transition towards longer horizons after this one.

**elchulito89** • 1y ago

I've done this before with the yang zhang volatility and also found some success.

**Mr-Zenor** • 1y ago

Awesome details right there!

**Passionate_Monkey** • 1y ago

Thankyou for this post.

**Darth_Casiuss** • 1y ago

Great post

**kalapur** • 1y ago

thank you, this is immensely useful!

**WARBUD_** • 1y ago

Nice post..



 **Opening-Ad2995** • 1y ago

Thanks for the great post! It's very cool to see some concrete discussion on this topic.

I haven't deployed the concept of meta labeling live, yet. I have been looking into how to apply it to a few strategies I have though. I was hoping you might have some thoughts on one line of questioning I have.

How do partition signals/features between your strategy and your ensemble meta labeling model?

I imagine many sensible features to model for the later meta labeling model could be incorporated as filter signals into the strategy itself. It seems there may often be some arbitrariness to this split.

Should you repeat features from your strategy in your meta labeling model? Avoid them? Move them out of your strategy so they are available for the model?

I'm not asking for specifics on your work, but conceptually, how to think about what belongs as a "signal in your strategy" vs. "feature in your meta labeling model"?

Thanks again, I've found your post incredibly interesting.



 **Yocurt OP** • 1y ago

Great question!

I typically fully develop the underlying strategy with all rules and features it needs before starting the meta labeling process at all. If I later see a feature in the meta labeling model has an extremely high feature importance score within the base models, I may go back and see how I can integrate that feature manually within the hard-coded strategy rules, but only follow through with it if there's a clear benefit from adding it to the actual strategy.

For the features to use for the meta labeling model, I make sure to use every single feature the underlying model used as features in every single base model for the ML as well. I think it's important for each model to have those features the underlying strategy uses to generate the signals. Then to get some diversity within the ML models, I add at least slightly different sets of features to each of the models that the underlying strategy does not use at all.

But to answer your question, I would repeat the features from your strategy into each ML model, and then on expand on that base set for each model.

The only one that wouldn't use these features is the final ensemble model, this one should just use the probability predictions from each of the base models for every trade, so one feature that will be a value between 0-1 for each of the base ML models.

 **Sure_Veterinarian_90** • 1y ago

My strategy (paper trading right now, going live soon) is a mix between an RL model with SL suggestions as features. I independently trained the supervised model to make suggestions whether (given indicators) a particular point in time is a good sell/buy moment. The trading agent is right now training with live fed data, and among financial features, there's the supervised model giving suggestions as quality (entry/exit) % It's been a week and still profitable, it is trading on BTC with coinbase ONE, because of 0% fee under 10k per month.

 **JurassicBarf** • 1y ago

Thanks for sharing, Existing edge could mean literally annotating times when a trade would've given 5 or 1 tick gain ? And using that as label



 **Yocurt OP** • 1y ago

By existing edge I mean more like a backtest that over a large number of individual trades, there is a consistent trend upwards in net profit. That could be a mix of different gains, but over time it shows repeatable results. But the underlying strategy should have at least 1000+ trades to have some confidence that the results are actually statistically significant.

Unless you're simulating slippage perfectly, I'd aim to use it on strategies that have at least 2-3 tick wins at a minimum. Every single trade gets its own individual label based on the metric you're using, even the losing trades - they're just as important for the model to learn what to avoid!

When deciding how to label each, you could label the winners (1) as trades with > 1 tick profit or > 5 tick profit based on your example. It's up to your discretion for what you want the model to interpret as a "good trade" that it should be looking to take.



 **JurassicBarf** • 1y ago

Yes, I've turned to multi-label classification.. +5, +10, -5, -10... the only thing I question myself on is data overlap e.g. t-10 to t used to predict t+5, then should this whole t-10 to t+5 sample not be used again in the training set to avoid label leakage or we good ?

also I'd love to collab and share some ideas that might help you as well. If okay, we can DM.



 **MeringueAlarming3102** • 7mo ago • Edited 7mo ago

Hey man, how is the process coming along for you? I'm also looking for some others who are applying some of De Prado's methods since I'm pretty damn lonely and don't know any other guys in this space, let alone manual traders.



Not sure if you're also doing intraday trades or scalps, but the reality of overlap is something that's tripping me up as well, although a bit from a different perspective.



JurassicBarf • 7mo ago

Sure thing, happy to be of help. DMd you.



MeringueAlarming3102 • 7mo ago

Thanks I appreciate the quick reply. I didn't even know if my comment would show up lol. Trying to stay awake but if I can't I'll DM you back within 24 hours!



ImEthan_009 • 1y ago

Very interesting. Have your forward tested with and without the filtering? What are the results?



Yocurt OP • 1y ago • Edited 1y ago

Been forward testing this since September, so about 9 months? First 3 months the results with and without the filtering were roughly the same so started to lose some hope, however the last 6 months the filtered results have improved the strategy about the amount shown in these images. The markets been a bit crazy with everything going on recently though (unfortunately it's just an unavoidable truth with trading) so I'm taking it all with a grain of salt.

But all in all, the filtering has definitely helped. My drawdown with the normal strategy would have been 35% over that time, but with the meta labeling it was 23%, and the profit would have been 28%, but with the filtering it's 21%



ImEthan_009 • 1y ago

Cool. Do you trade in the stock market? Is it single-name or multiple equities?



Yocurt OP • 1y ago

I only have strategies for ES and NQ. The images in this post are from my ES strategy. The meta labeling models are completely separate for each strategy.

If you wanted to apply this to stocks and create a single meta labeling model that is trained on signals from multiple assets, you can use a categorical feature in every model that tells the model which asset the signal is for, so it can try to learn how that asset affects the results.

For example, if your underlying strategy generated 500 signals for TSLA and 500 signals for APPL, and had a 5% win rate and 95% win rate on those two, respectively (a bit exaggerated just for the example), it would likely learn to skip the TSLA trades and take APPL trades.

You would still have all the other features as well for the ML models, just this categorical one in addition to those.



ImEthan_009 • 1y ago

Thanks that explains better I wasn't quite following. Hope your filtering works well. I rank the S&P 500 and Nasdaq 100 components (there are 516 stocks) based on momentum and mean reversion value. Finalised the parameters couple of years ago and have since done thousands of validations. Confidence as volatile as TSLA. But I've put money into it since 28 April 25, so far just beating the index but can be luck. Should take 5 years to validate.



shaonvq • 1y ago

What asset(s) are you trading in the graphic?



Yocurt OP • 1y ago

Those plots are all from my ES strategy



Professional_Bee8461 • 1y ago

Great post! I would really appreciate it if you could recommend some resources for beginners who want to learn algo trading.



Yocurt OP • 1y ago

Skip to main content

+ Create

1

**heygentlewhale** • 1y ago

This is really helpful! thank you

**heygentlewhale** • 1y ago

Any advice how to get started, currently I am only backtesting on TradingView

**Xbldev** • 1y ago

sorry for the noob question: does using a meta model slow you down? I imagine the time required to build features etc would cause loss of time? Assuming you are doing hft this could be problematic?

**Yocurt OP** • 8d ago

That's a good question. Building the pipeline for it certainly costs some time. But for during live trading, it really doesn't increase the execution latency much, but for hft it may be enough to matter. I don't do hft so not too sure.

Each time you run your code to retrain the models, it can take awhile. When you go live though, you just compile the trained model that has all the weights and everything baked in, then when you pass the feature values in, its just predicting one value which is typically basically instant

**Xbldev** • 8d ago

hi thanks for replying 😊 so you are not doing HFT? what's your timeframe if you don't mind me asking?

**VeiledTrader** • 1y ago

Great post! Smart way to visualize the results with and without xgb

**Remarkable-Ant-753** • 1y ago

That's a very detailed explanation. Thanks for that.

Just needed clarity about Meta labeling. Isn't the Meta labeling used for determining the size of the bet?

How did you strategize it into determining the type of the trade?

**Prize-Sport-3328** • 1y ago

Algorithmic Trader

Great post! Thanks for sharing

**beast_of_iit** • 1y ago

Yeaah bro, XG boost works most of the time, easy at inference time also

**Ok-Negotiation5058** • 10mo ago

best post I ever seen in the sub, thank you so much for sharing

**imranaalam786** • 10mo ago

THANK YOU.

it would be great if you could share some code. excluding your strategy.

===

Comprehensive Plan for Implementing Meta-Labeling

[+ Create](#)

Phase 1: Foundation & Strategy Selection

The entire process fails if the underlying strategy is flawed. The goal here is to select a robust primary strategy before any machine learning is applied.

1. **Adopt the Core Philosophy:** Understand that you are not using ML to find an edge. You are using it to amplify an existing, proven edge.
2. **Develop or Identify a Primary Strategy:** Create a simple, rules-based strategy (e.g., trend-following, mean-reversion).
3. **Validate the Primary Strategy's Edge:** Before proceeding, your base strategy must meet strict performance and robustness criteria. Based on the author's recommendations, your strategy should have:
 - **Profit Factor:** At least **1.07** after accounting for realistic costs.
 - **Backtest Period:** A minimum of **5 years** of historical data.
 - **Statistical Significance:** At least **2,500** total trades to ensure the results aren't due to luck.
 - **Consistency:** Be profitable in approximately **80%** of the years tested.
 - **Risk Management:** The maximum drawdown should not exceed the average yearly profit.
 - **Accurate Cost Simulation:** Your backtest engine must accurately model slippage and fees from the start.
4. **Guard Against Initial Overfitting:** Be extremely critical of your base strategy. Ensure its performance is not a result of curve-fitting to historical data. Use out-of-sample data and walk-forward analysis on the base strategy itself before even considering meta-labeling.

18nebula • 10mo ago • Edited 10mo ago

[deleted] • 5mo ago

Comment deleted by user



Yocurt OP • 5mo ago

Yes that's great, it definitely should! The more uncorrelated your features and your base models are the better

TonyGTO • 1y ago • Edited 1y ago

Look, if a machine learning model can't spot patterns in your price time series, it's probably because the dataset's too thin. You need more context, like time series from other instruments, macro indicators, fundamentals, the whole nine yards.

You've gotta train it across every financial asset imaginable, then run genetic algorithms to weed out the top performers.

[deleted] • 1y ago

Comment deleted by user



Yocurt OP • 1y ago

I'd think for a lot of people - other than you of course smarty pants!

Ok-Swan-9842 • 1y ago

try the quant system at cromcall.com very simply easy to follow

Emotional_Buy_6712 • 1d ago

Thank you, very insightfull!!! love it